

# Projekt SK– Symulator UAR

## Etap 3

Skład grupy:

- Paweł Nagórzański
- Kacper Chmura

# Wykrywanie opóźnień

Odbiór ramki z wartością regulowaną z sieci:

- MenadzerSieci::obsluzRamke
- MenadzerSymulacji::aktualizujZSieciObiekt
- Ustawienie zmiennej \_czy\_dane\_dotarly na true
- Wyzwolenie kolejnego taktu zegara symulacji
- MenadzerSymulacji::wykonajKrokSymulacji
- Wywołanie emit sygnałWydajnosci z argumentem \_czy\_dane\_dotarly
- Reset zmiennej \_czy\_dane\_dotarly na false dla następnego cyklu
- MainWindow::onSygnałWydajnosci
- Zmiana koloru kontrolki \_labelWydajnosci na zielony lub czerwony zależnie od otrzymanej wartości logicznej

MenadzerSieci::obsluzRamke

```
case TypRamki::SymulacjaObiekt: {  
    double czas, y;  
    deserializeSymulacjaObiekt(payload, czas, y);  
    _manager->aktualizujZSieciObiekt(y);  
    break;  
}  
void MenadzerSymulacji::aktualizujZSieciObiekt(double y) {  
    _ostatnia_wartosc_regulowana = y;  
    _czy_dane_dotarly = true;  
}
```

MenadzerSymulacji::wykonajKrokSymulacji

```
} else if (_trybPracy == TrybPracy::SiecRegulator) {  
  
    emit sygnałWydajnosci(_czy_dane_dotarly);  
    _czy_dane_dotarly = false; // Reset na kolejny cykl zegara  
  
    double uchyb = wartoscZadana - _ostatnia_wartosc_regulowana;  
    double u = _regulator.symuluj(uchyb);  
  
    emit wyslijRamkeRegulatora(czas, wartoscZadana, u);  
    emit noweDataReady(czas, wartoscZadana, _ostatnia_wartosc_regulowana, u, uchyb, _regulator.getOstatnieSkładowe());  
}
```

[Regulator/Serwer] Połączono z: 169.254.187.249 (port 0)



# Rozwiązanie problemów z opóźnieniami

- Brak blokowania symulacji. Zegar QTimer pracuje bez przerw, obliczenia nie czekają na sieć.
- W przypadku braku nowej próbki regulator wykorzystuje ostatnią znaną wartość.
- Ciągłość działania. Wykresy nie przerywają się, a algorytm PID kontynuuje pracę.
- Optymalizacja transmisji. Serializacja binarna zmniejsza rozmiar ramek do minimum.
- Ręczna korekta. Można w dowolnej chwili wydłużyć interwał taktowania symulacji.

# Końcowa wersja protokołu sieciowego

## Serializacja:

- Zmiana parametrów
- Serializacja do postaci binarnej (stała kolejność)
- Budowanie ramki protokołu
- Wysłanie ramki
- Odbiór ramki, weryfikacja CRC i deserializacja

```
QByteArray serializePID(const RegulatorPID& pid, quint32 timestamp) {
    QByteArray buf;
    QDataStream out(&buf, QIODevice::WriteOnly);
    out.setVersion(QDataStream::Qt_6_0);

    out << timestamp
        << pid.getKp() << pid.getTi() << pid.getTd() << pid.getTp()
        << pid.getUMin() << pid.getUMax()
        << (quint32)pid.getLiczCalk();

    return buf;
}
```

```
void MyTCPServer::wyslijRamke(quint8 typ, const QByteArray& payload, int numCli) {
    if (numCli < 0 || numCli >= m_clients.length()) return;

    QByteArray frame;
    QDataStream out(&frame, QIODevice::WriteOnly);
    out.setByteOrder(QDataStream::BigEndian);

    out << (quint8)0xAA;           //START
    out << typ;                     //TYP
    out << (quint16)payload.size(); //DLUGOSC

    frame.append(payload);         //DANE

    quint16 crc = 0;
    for (unsigned char b : frame) crc += b; //CRC
    QDataStream crcOut(&frame, QIODevice::Append);
    crcOut.setByteOrder(QDataStream::BigEndian);
    crcOut << crc;

    m_clients.at(numCli)->write(frame);
}
```

# Końcowa wersja protokołu sieciowego

| Typ ramki          | Start | Typ ramki | Długość                            | Dane                                                      | CRC                       |
|--------------------|-------|-----------|------------------------------------|-----------------------------------------------------------|---------------------------|
| Długość            | 1B    | 1B        | 2B                                 | zależy                                                    | 2B                        |
| PID                | 0xAA  | 0x01      | 56B<br>(binarnie)                  | timestamp-Kp-Ti-Td-Tp-Umin-Umax-TrybCalk                  | Suma Bajtów<br>(binarnie) |
| ARX                |       | 0x02      | Zależy do<br>długości<br>A[] i B[] | timestamp-A[]-B[]-k-szum-ograniczenie-Umin-Umax-Ymin-Ymax |                           |
| Generator          |       | 0x03      | 36B<br>(binarnie)                  | Typ-amplituda-okres-wypełnienie- StałaSkładowa            |                           |
| InfoPolaczenia     |       | 0x04      | 22B-38B<br>(binarnie)              | rola, lokalneIP                                           |                           |
| SymulacjaRegulator |       | 0x06      | 24B<br>(binarnie)                  | czas, w, u                                                |                           |
| SymulacjaObiekt    |       | 0x07      | 16B<br>(binarnie)                  | czas, y                                                   |                           |
| Reset              |       | 0x08      | 0B                                 | ---                                                       |                           |